

Convolutional Neural Networks

<http://cs231n.stanford.edu/>

Fei-Fei Li & Andrej Karpathy & Justin Johnson

Lecture 7 - 1

27 Jan 2016

743

A bit of history:

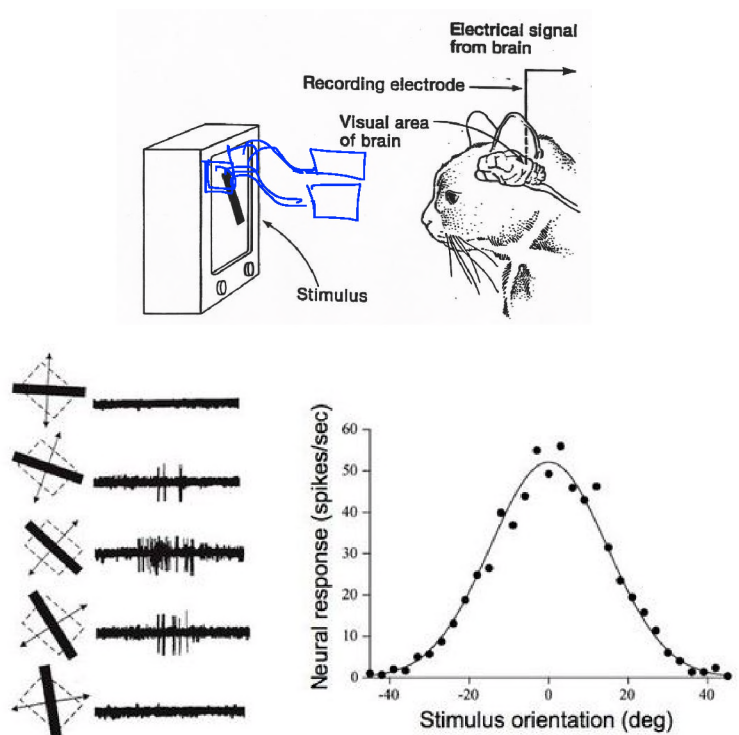
**Hubel & Wiesel,
1959**

RECEPTIVE FIELDS OF SINGLE
NEURONES IN
THE CAT'S STRIATE CORTEX

1962

RECEPTIVE FIELDS, BINOCULAR
INTERACTION
AND FUNCTIONAL ARCHITECTURE IN
THE CAT'S VISUAL CORTEX

1968...



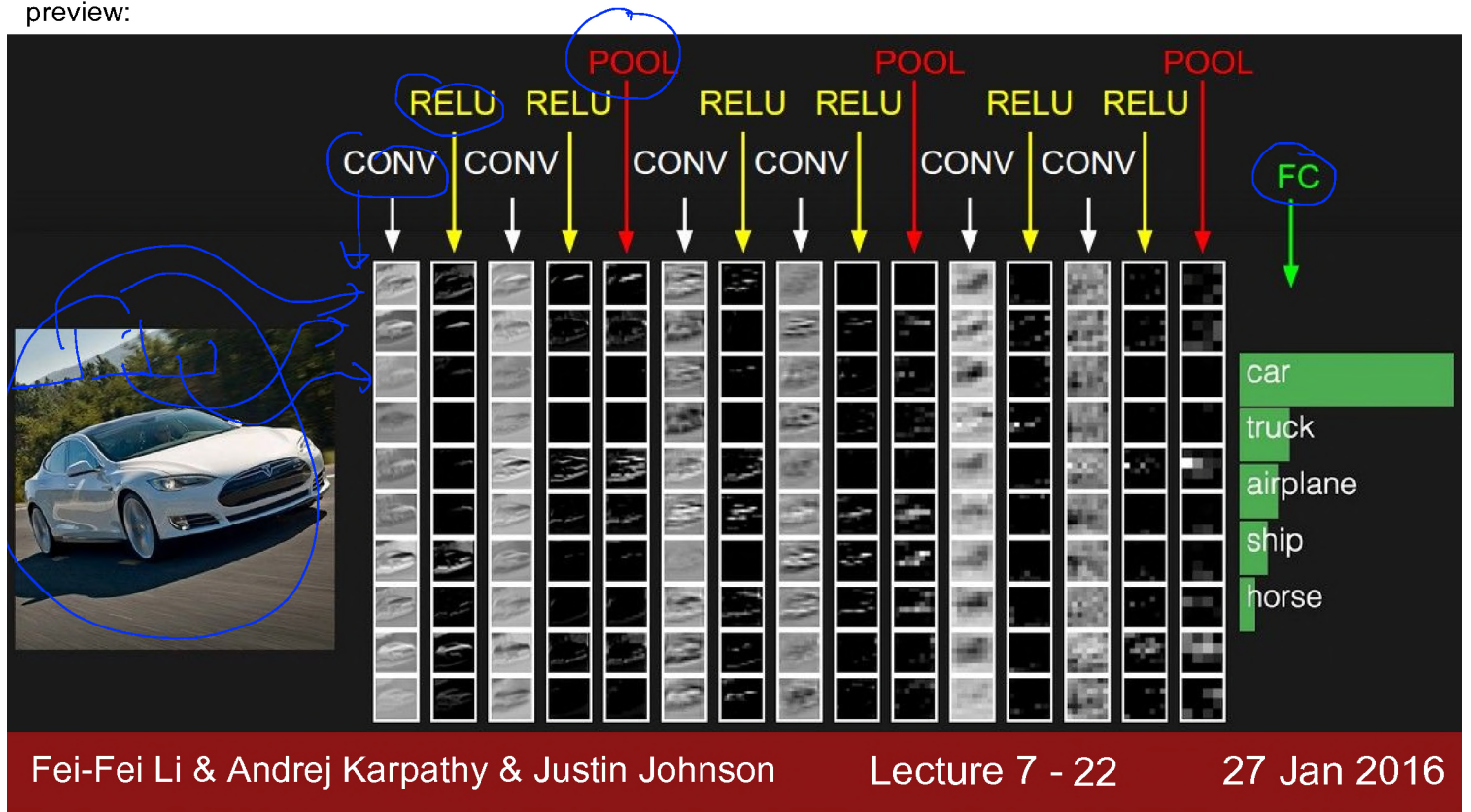
Fei-Fei Li & Andrej Karpathy & Justin Johnson

Lecture 7 - 7

27 Jan 2016

744

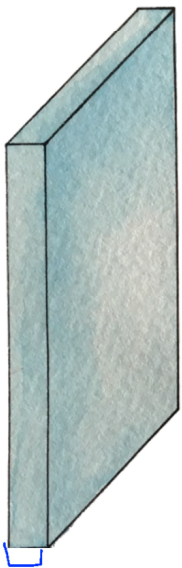
preview:



745

745

Start with an image (width x height x depth)

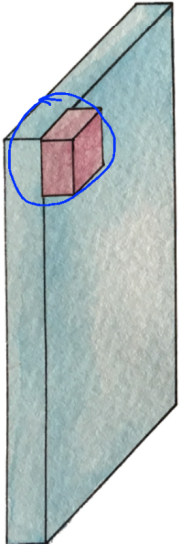


32x32x3 image

746

746

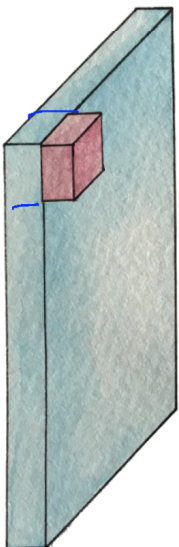
Let's focus on a small area only



32x32x3 image

747

Let's focus on a small area only (5x5x3)



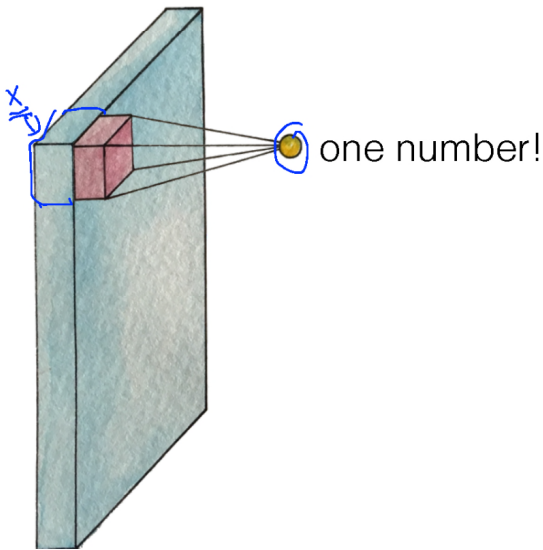
32x32x3 image



5x5x3 filter

748

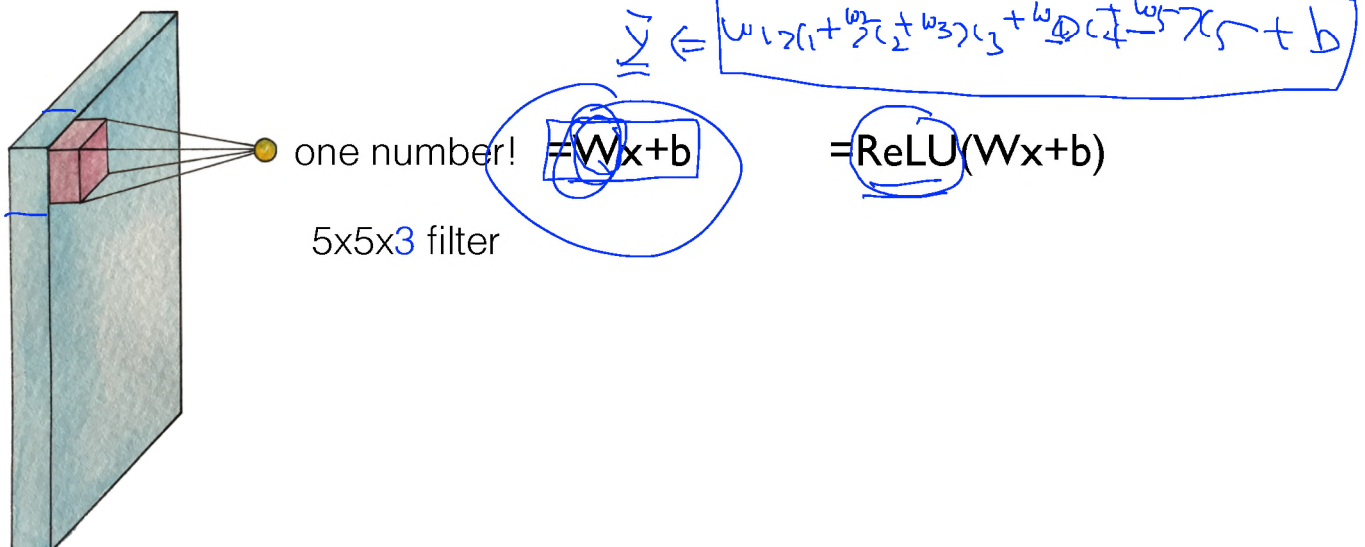
Get one number using the filter



32x32x3 image

749

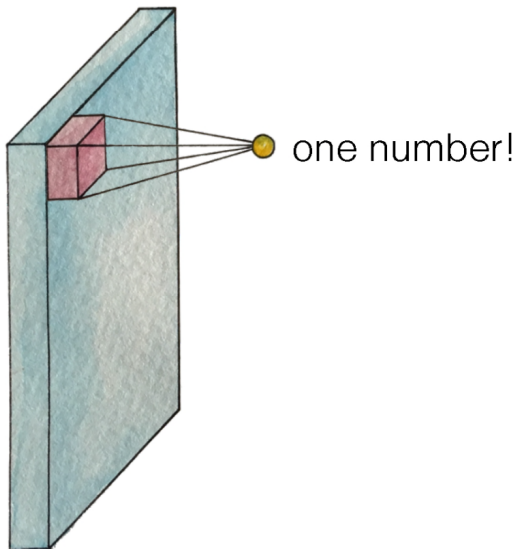
Get one number using the filter



32x32x3 image

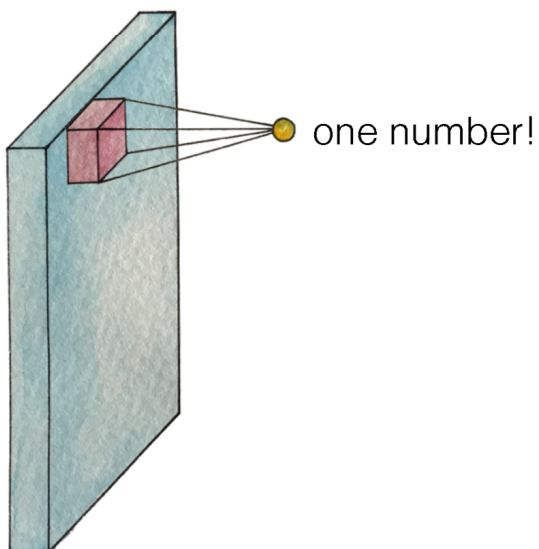
750

Let's look at other areas with the same filter (w)



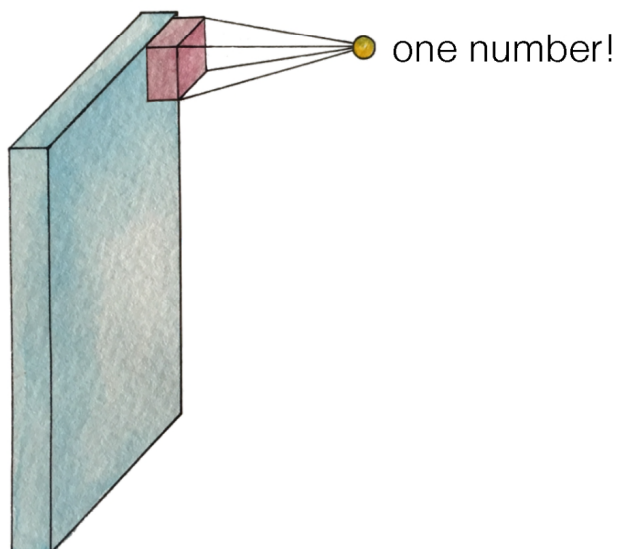
32x32x3 image

Let's look at other areas with the same filter (w)



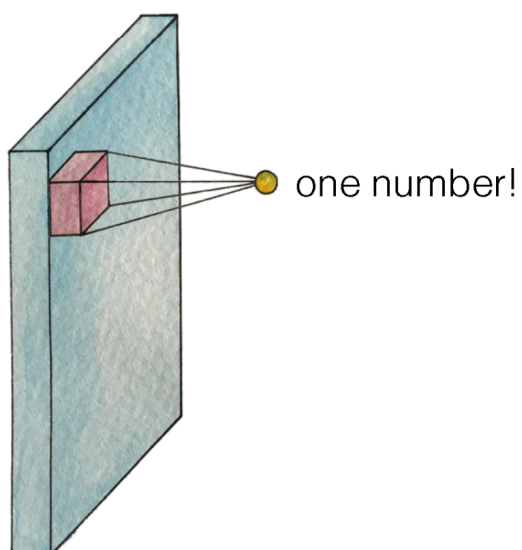
32x32x3 image

Let's look at other areas with the same filter (w)



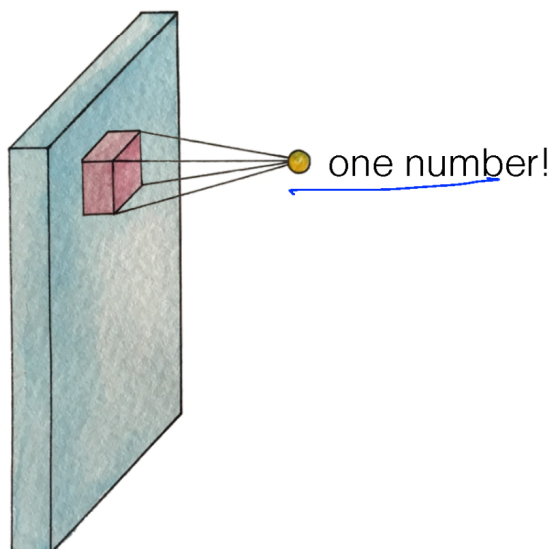
32x32x3 image

Let's look at other areas with the same filter (w)



32x32x3 image

Let's look at other areas with the same filter (w)



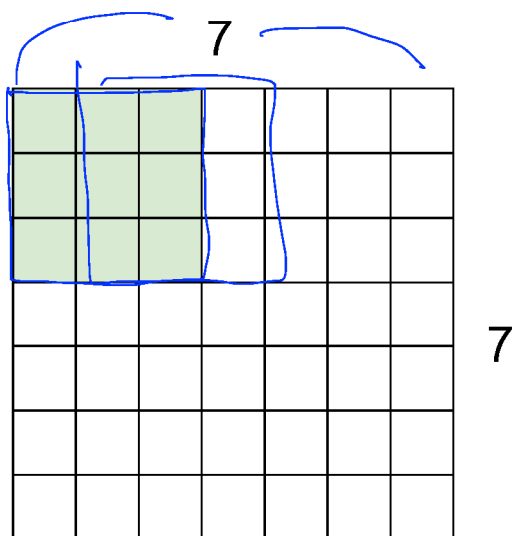
32x32x3 image

How many numbers
can we get?

755

755

A closer look at spatial dimensions:

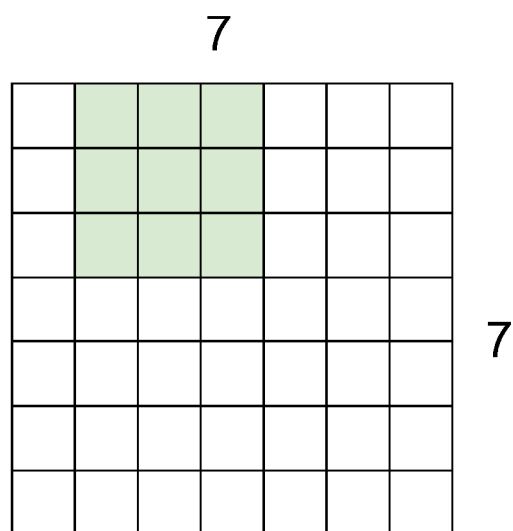


7x7 input (spatially)
assume 3x3 filter

756

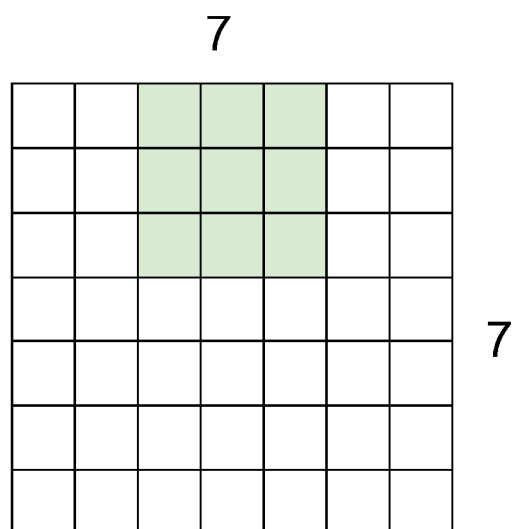
756

A closer look at spatial dimensions:



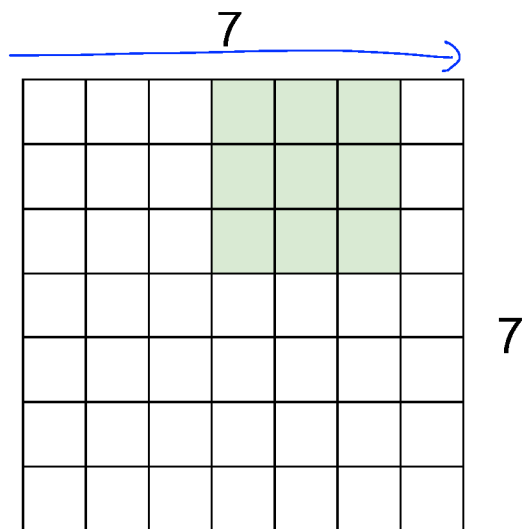
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

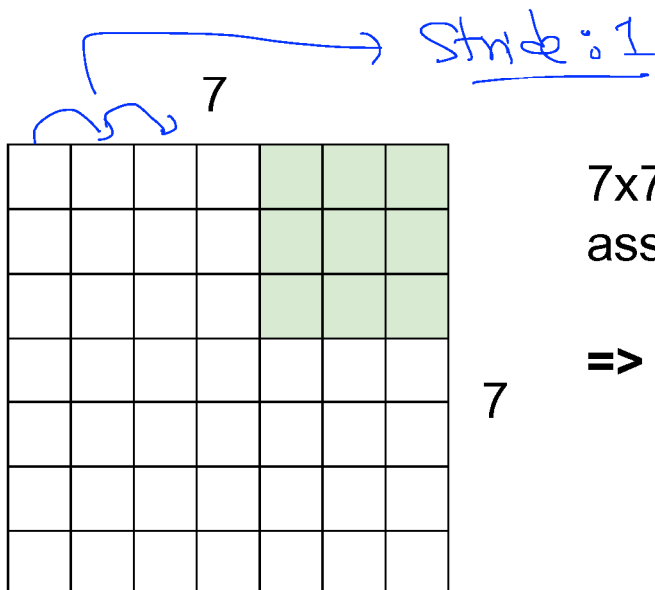
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

5x5

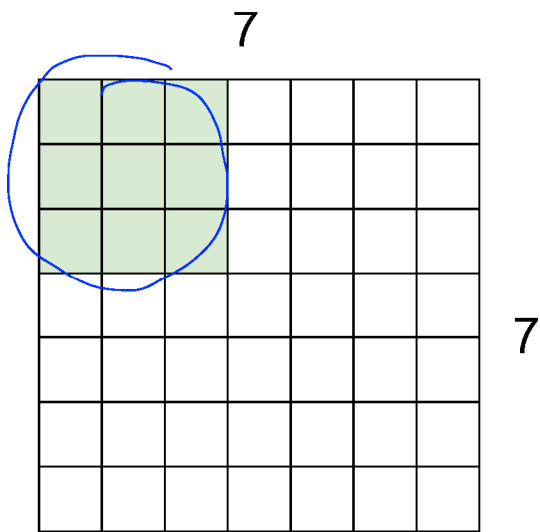
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

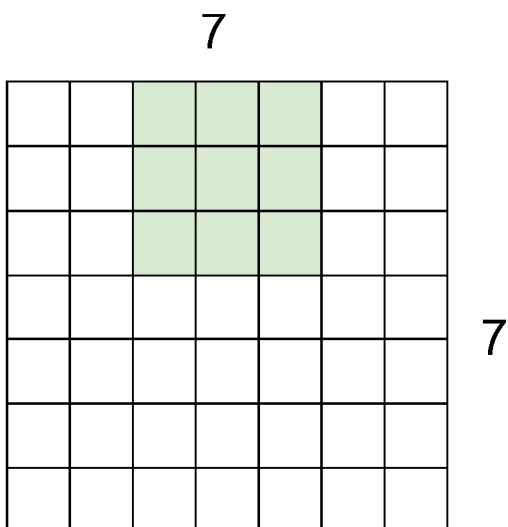
=> 5x5 output

A closer look at spatial dimensions:



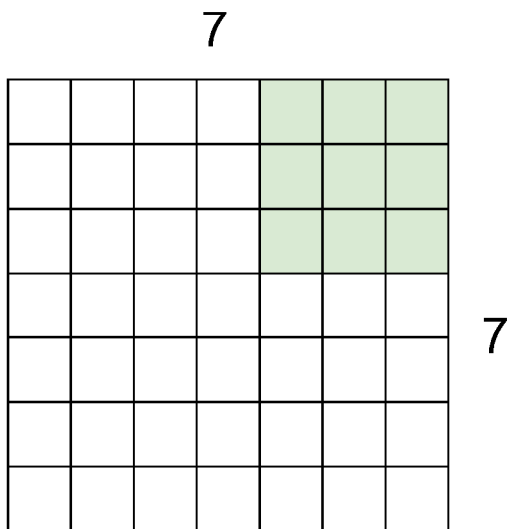
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:

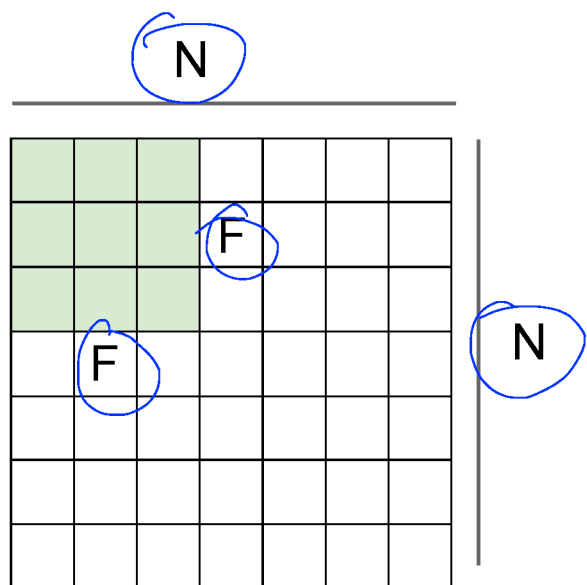


7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!



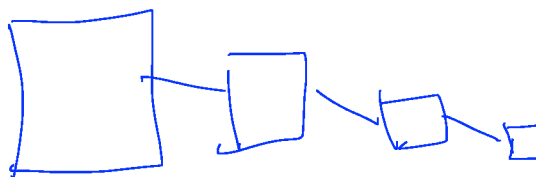
Output size:
 $(N - F) / \text{stride} + 1$

e.g. $N = 7$, $F = 3$:

stride 1 => $(7 - 3) / 1 + 1 = 5$

stride 2 => $(7 - 3) / 2 + 1 = 3$

~~stride 3 => $(7 - 3) / 3 + 1 = 2.33$: \~~



In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7
3x3 filter, applied with stride 1
pad with 1 pixel border => what is the output?

$7 \times 7 \rightarrow 9 \times 9$
pad

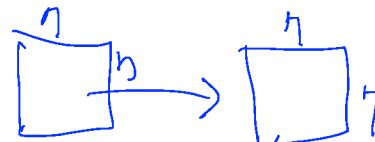
(recall:)
 $(N - F) / \text{stride} + 1$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7
3x3 filter, applied with stride 1
pad with 1 pixel border => what is the output?

7x7 output!



In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input **7x7**

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

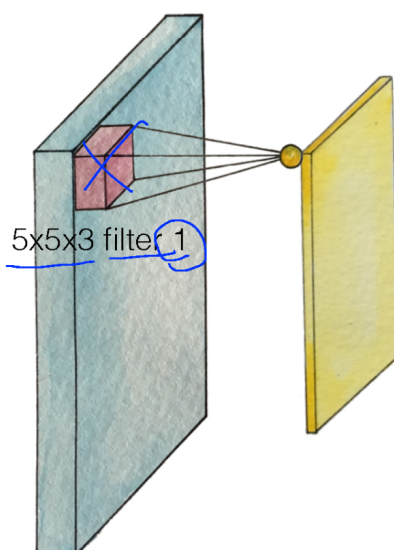
in general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$. (will preserve size spatially)

e.g. $F = 3 \Rightarrow$ zero pad with 1

$F = 5 \Rightarrow$ zero pad with 2

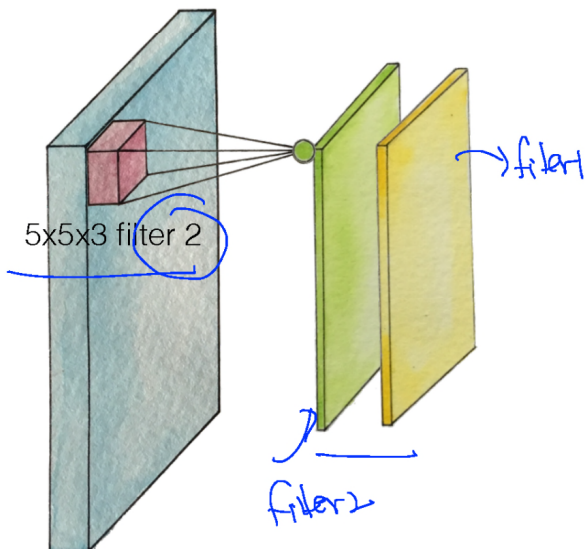
$F = 7 \Rightarrow$ zero pad with 3

Swiping the entire image



32x32x3 image

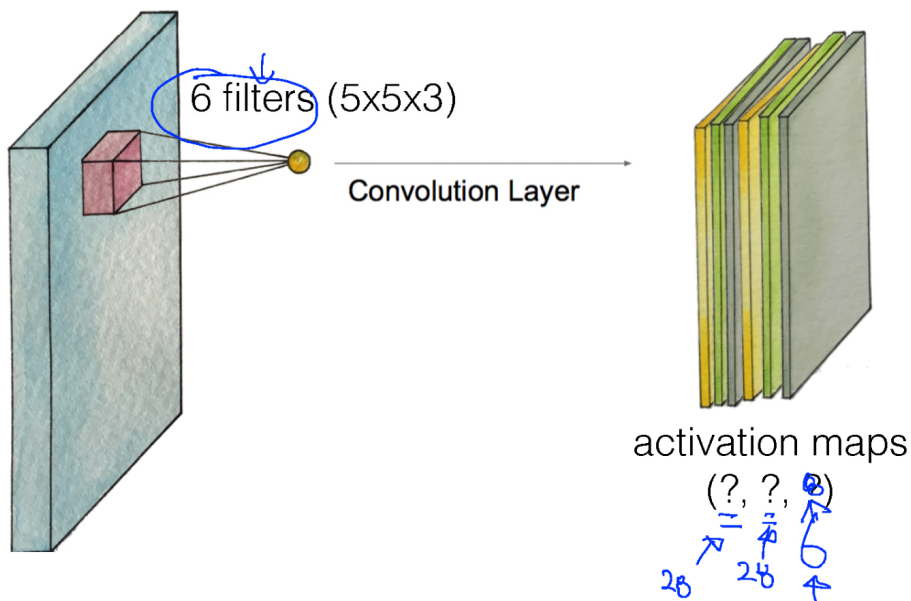
Swiping the entire image



32x32x3 image

769

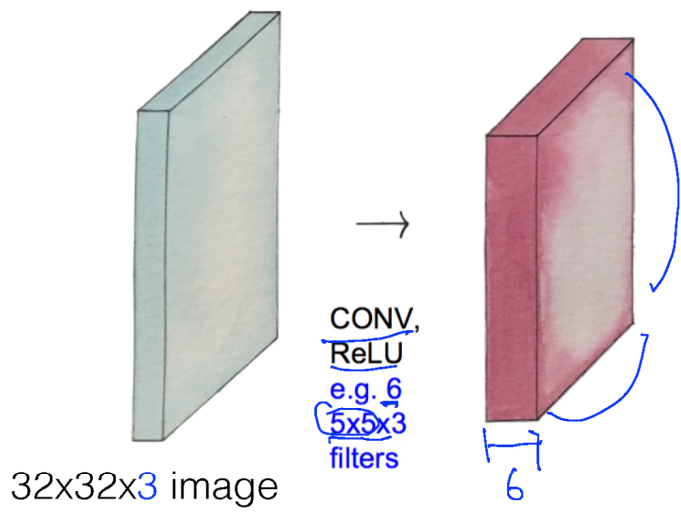
Swiping the entire image



32x32x3 image

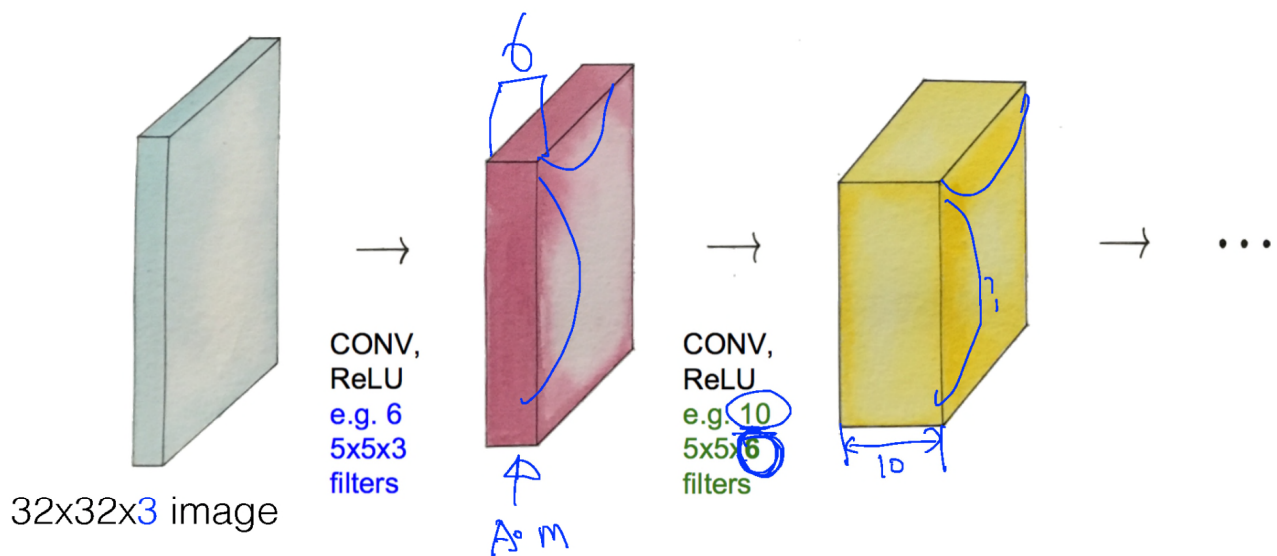
770

Convolution layers



771

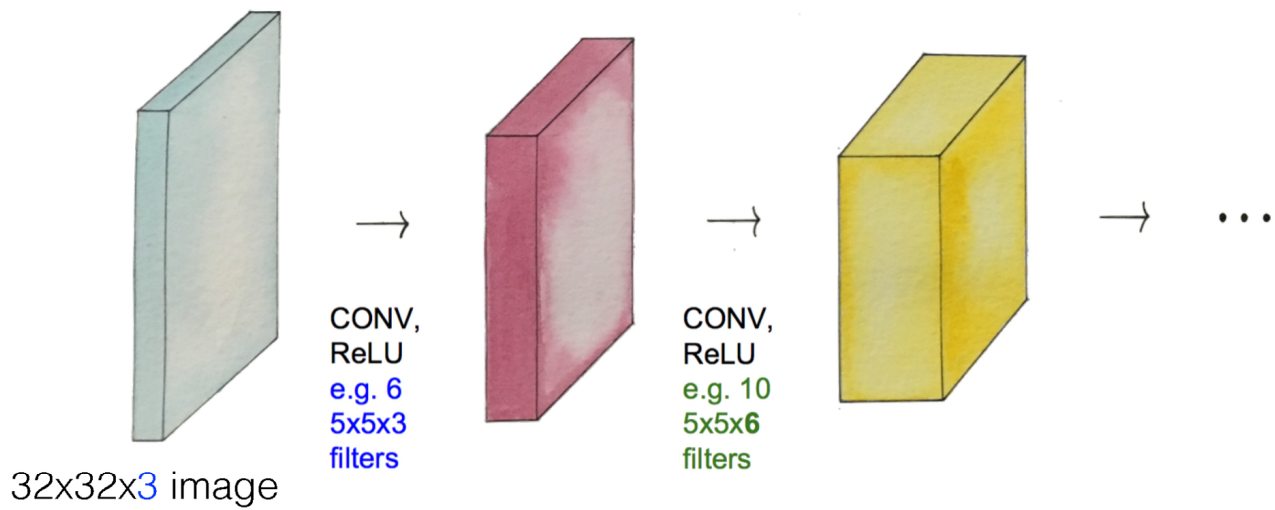
Convolution layers



772

Convolution layers

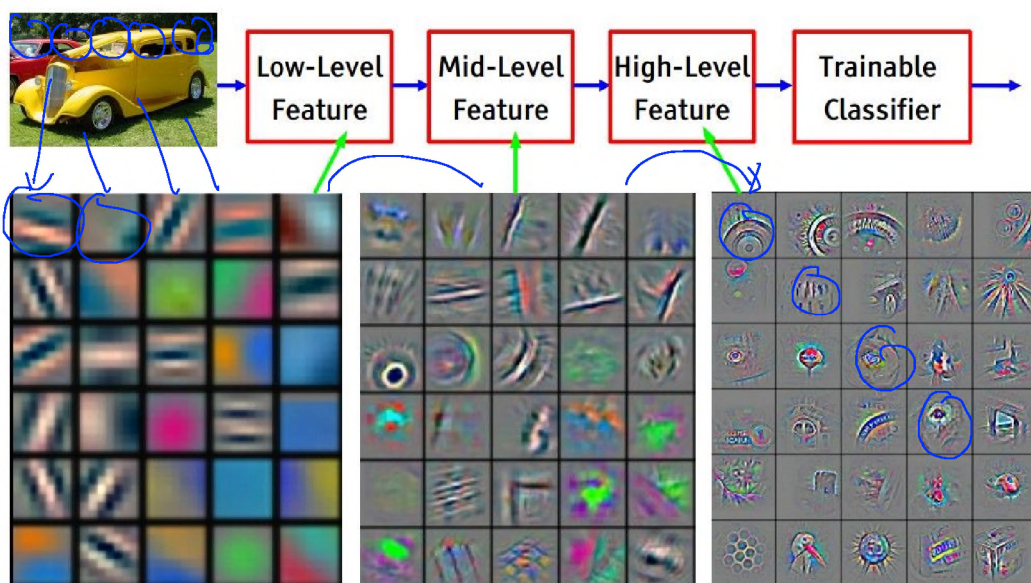
How many weight variables? How to set them?



773

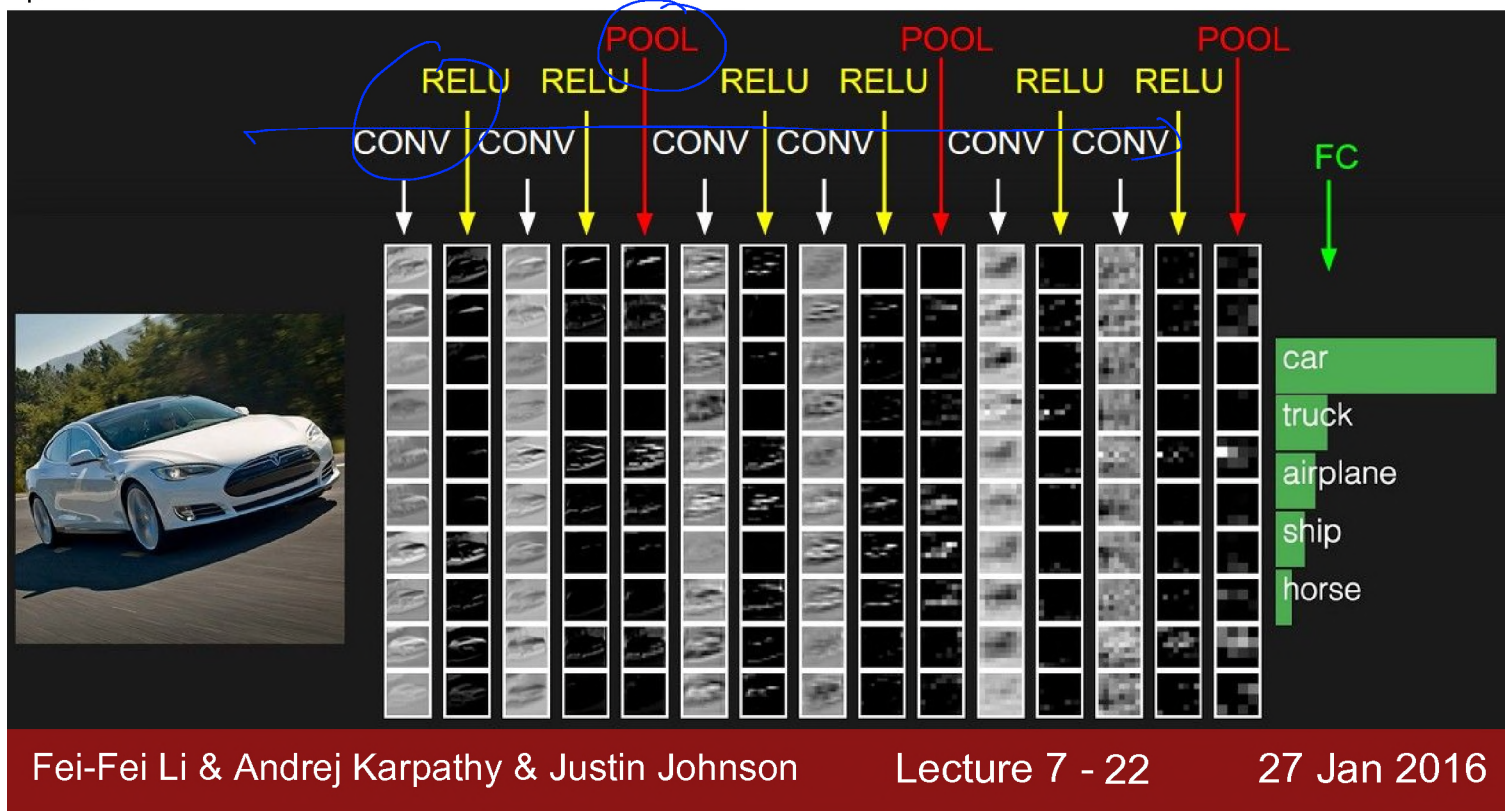
Preview

[From recent Yann LeCun slides]



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

preview:



775

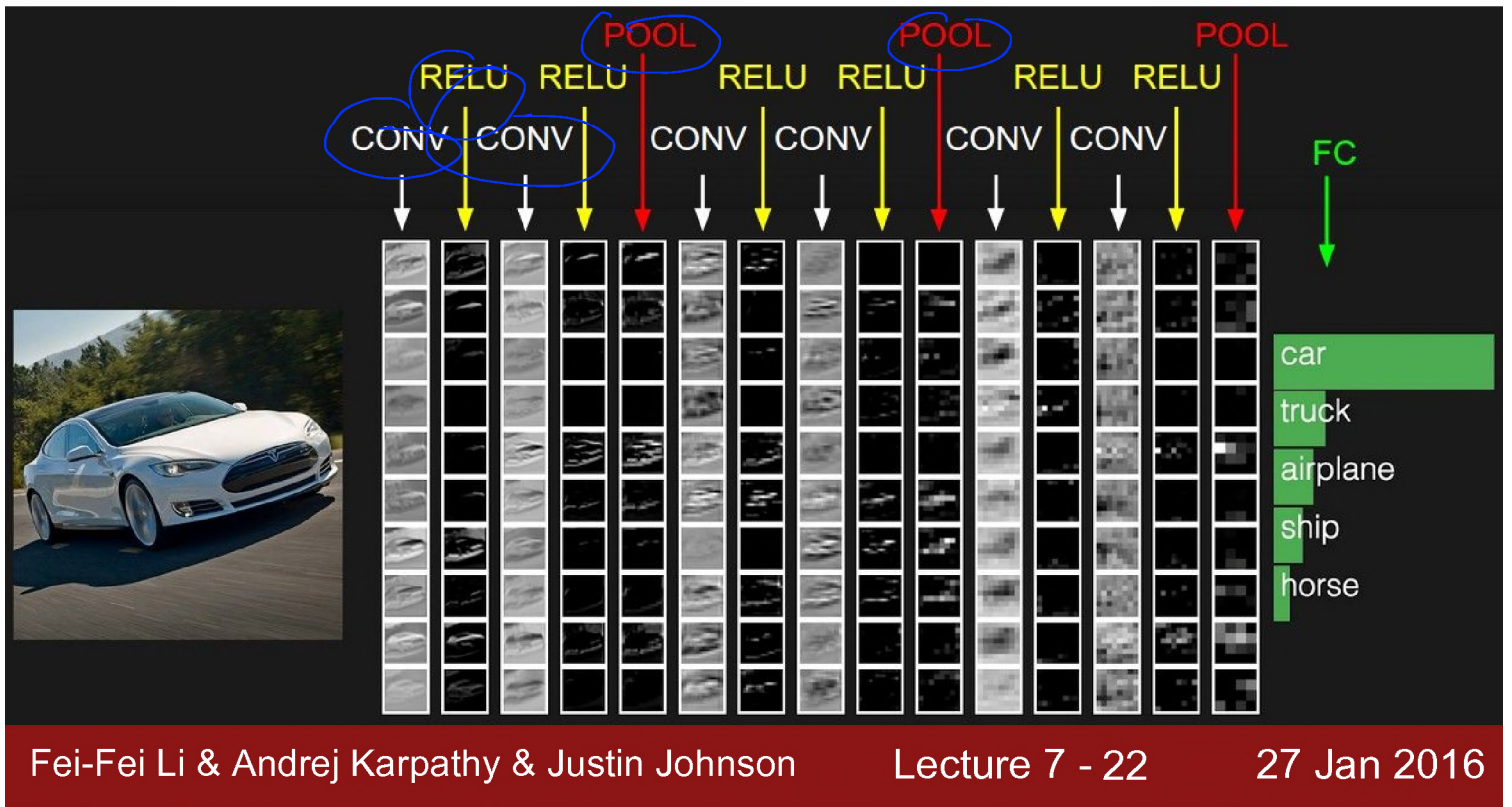
Lecture 11-2

CNN introduction: Max pooling and others

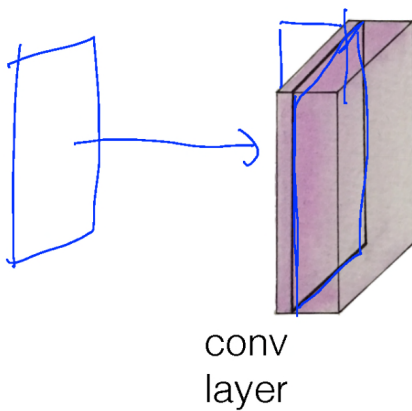
Sung Kim <hunkim+mr@gmail.com>

776

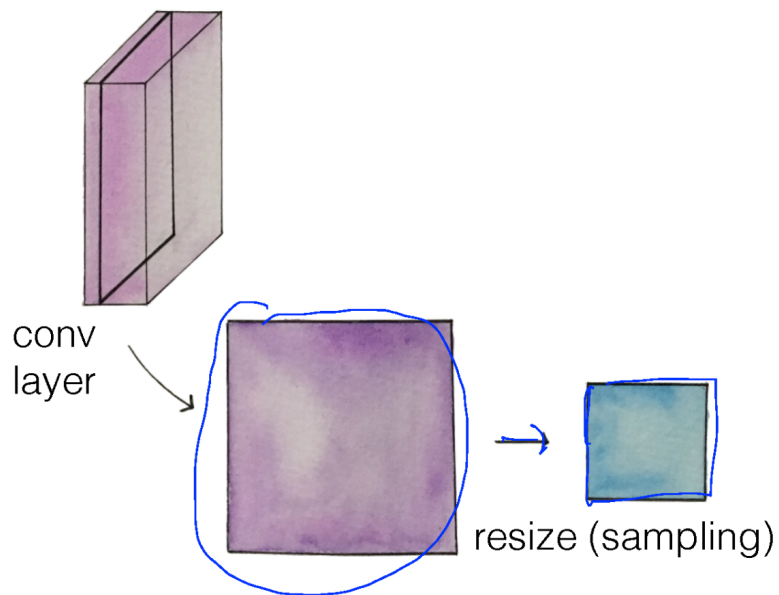
preview:



Pooling layer (sampling)

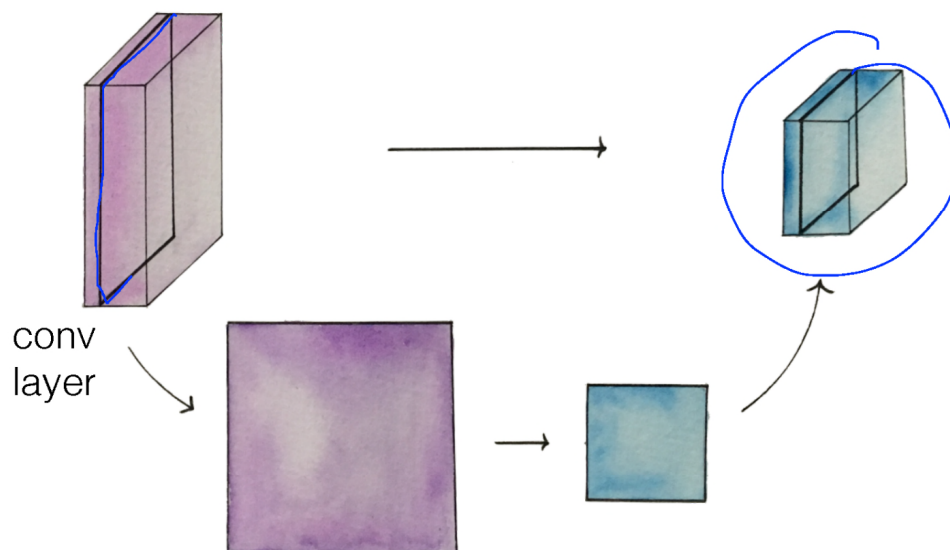


Pooling layer (sampling)



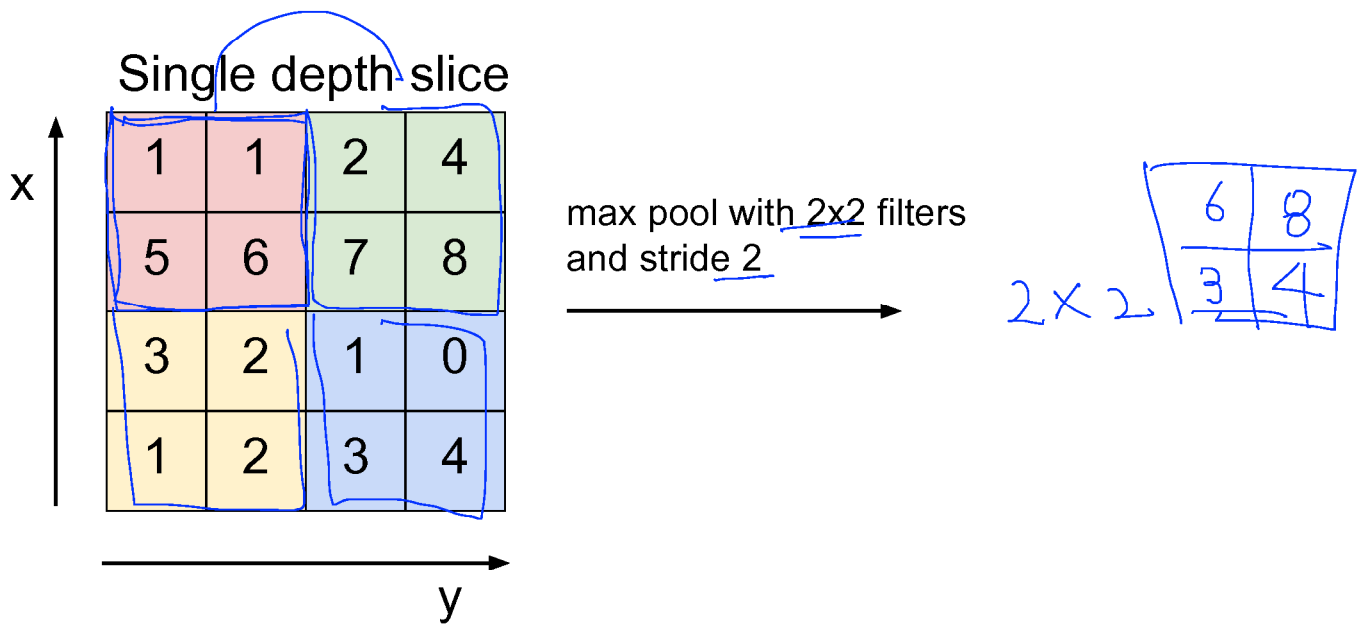
779

Pooling layer (sampling)

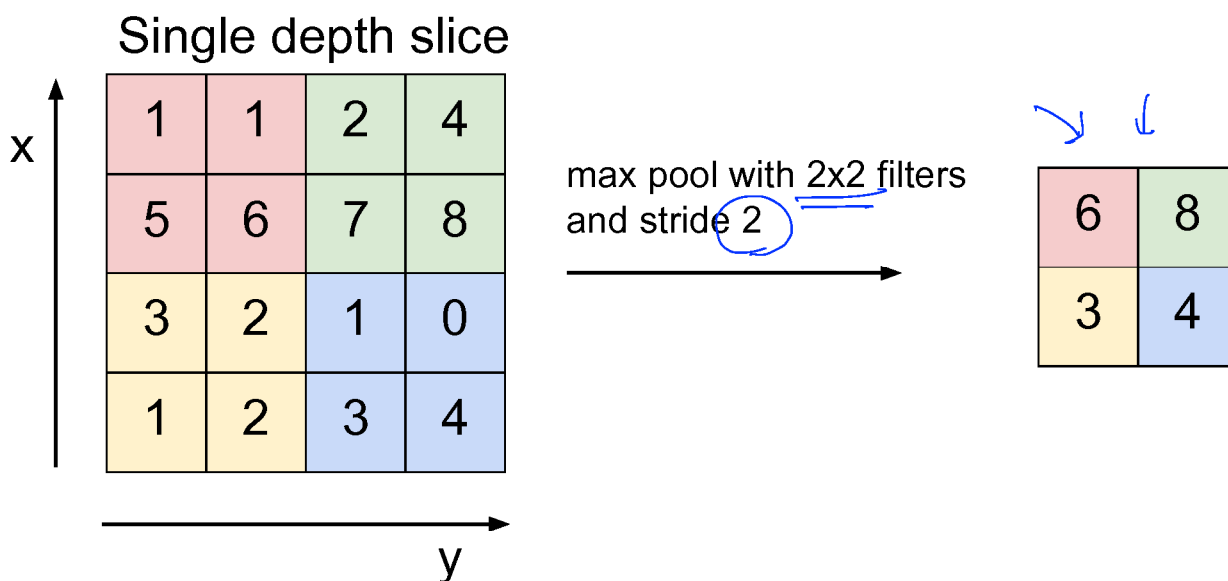


780

MAX POOLING

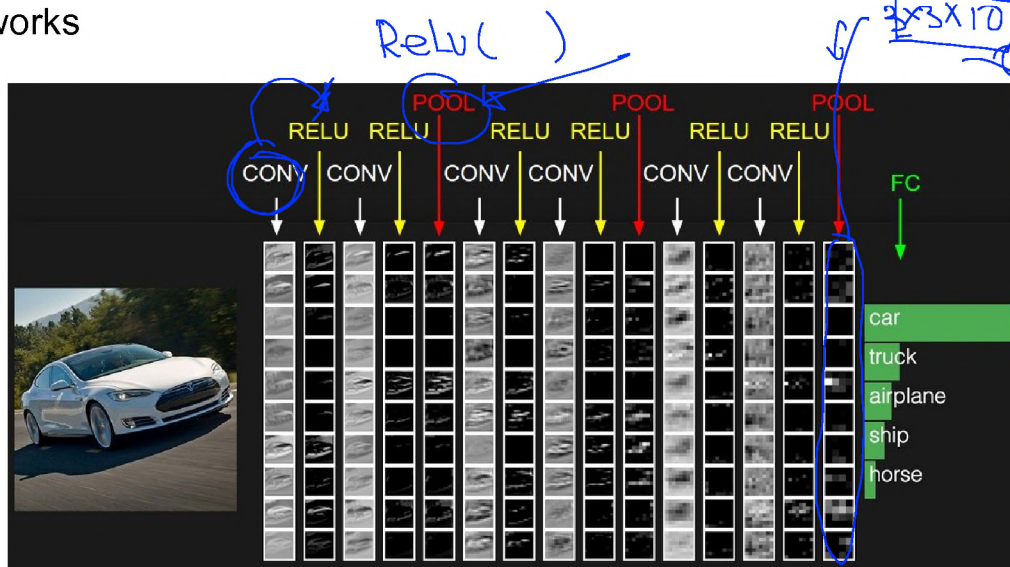


MAX POOLING



Fully Connected Layer (FC layer)

- Contains neurons that connect to the entire input volume, as in ordinary Neural Networks



[ConvNetJS demo: training on CIFAR-10]



<http://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html>

Convolutional Neural Networks

<http://cs231n.stanford.edu/>

Fei-Fei Li & Andrej Karpathy & Justin Johnson

Lecture 7 - 1

27 Jan 2016

785

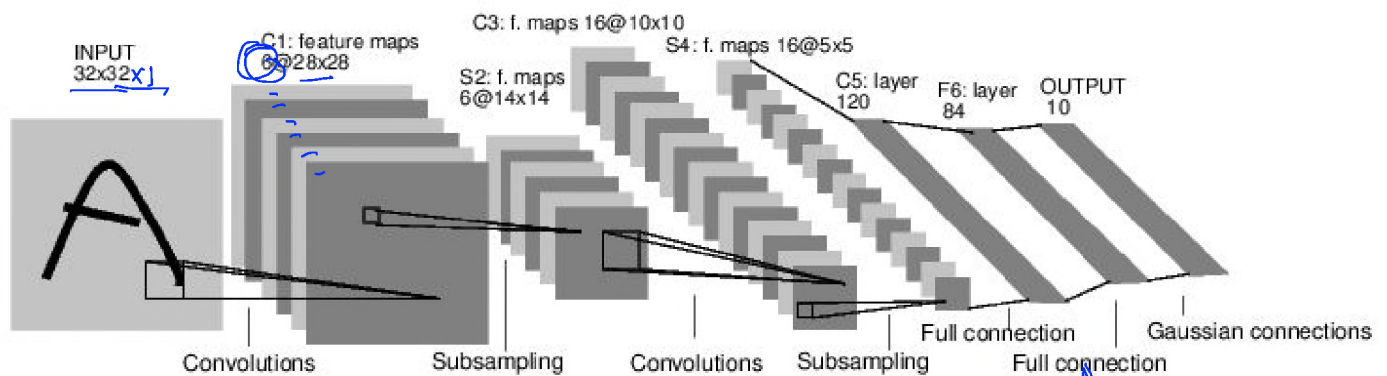
9.8 합성곱 신경망 모델의 성공

- ◆ 고전적인 손글씨 데이터베이스 뿐만 아니라 여러 종류의 사물을 인식하도록 하기 위하여 대규모 이미지 데이터베이스를 구축하는 작업은 스탠퍼드 대학교의 페이페이리^{Fei-Fei Li}교수가 주관
- ◆ 2만2천개의 범주로 구분된 1,500만장의 방대한 이미지로 구성된 이미지넷^{ImageNet} 데이터베이스는 오늘날의 딥러닝 기술을 발전시키는데 크게 기여



Case Study: LeNet-5

[LeCun et al., 1998]



Conv filters were 5x5, applied at stride 1
Subsampling (Pooling) layers were 2x2 applied at stride 2
i.e. architecture is [CONV-POOL-CONV-POOL-CONV-FC]

Fei-Fei Li & Andrej Karpathy & Justin Johnson

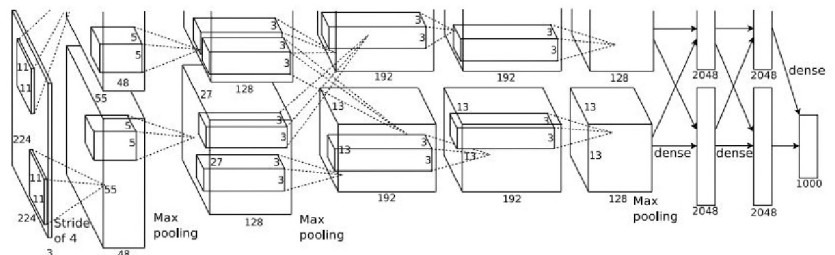
Lecture 7 - 60

27 Jan 2016

787

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

First layer (CONV1): 96 11x11 filters applied at stride 4

=>

Output volume [55x55x96]

Parameters: $(11 \times 11 \times 3) \times 96 =$ 35K

Fei-Fei Li & Andrej Karpathy & Justin Johnson

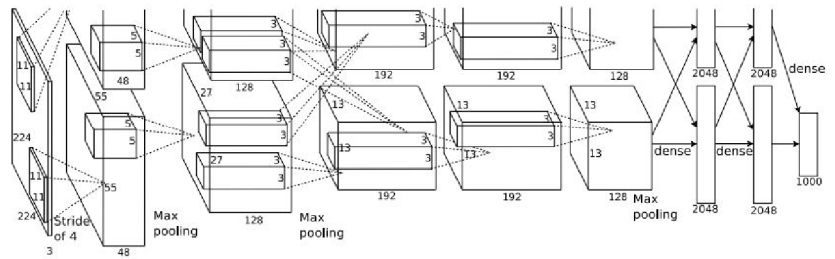
Lecture 7 - 63

27 Jan 2016

788

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

After CONV1: 55x55x96

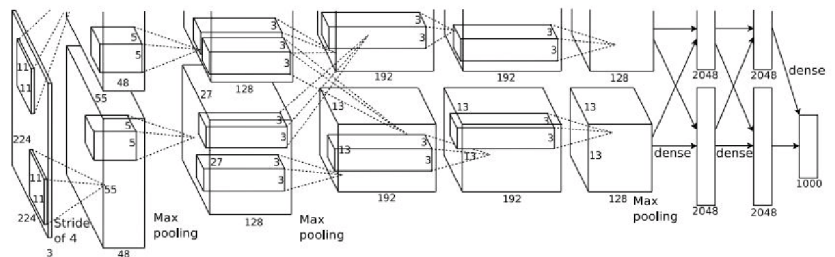
Second layer (POOL1): 3x3 filters applied at stride 2

Output volume: 27x27x96

Parameters: 0!

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

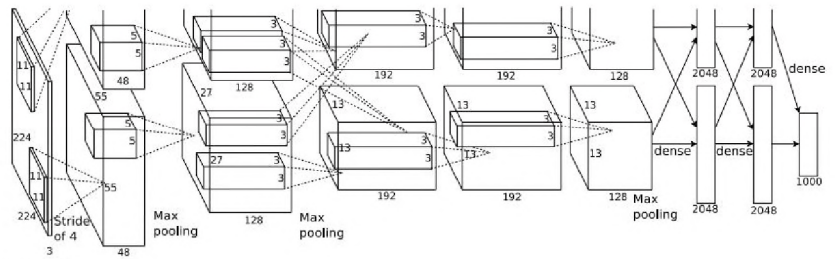
After CONV1: 55x55x96

After POOL1: 27x27x96

...

Case Study: AlexNet

[Krizhevsky et al. 2012]



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

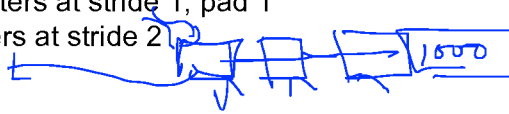
[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)



Fei-Fei Li & Andrej Karpathy & Justin Johnson

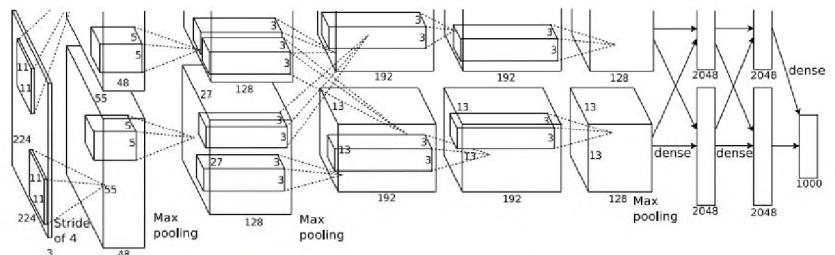
Lecture 7 - 68

27 Jan 2016

791

Case Study: AlexNet

[Krizhevsky et al. 2012]



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)

Details/Retrospectives:

- first use of **ReLU**
- used **Norm layers** (not common anymore)
- heavy data augmentation
- dropout 0.5
- batch size 128
- SGD Momentum 0.9
- Learning rate 1e-2, reduced by 10 manually when val accuracy plateaus
- L2 weight decay 5e-4
- 7 CNN **ensemble** 18.2% -> 15.4%

Fei-Fei Li & Andrej Karpathy & Justin Johnson

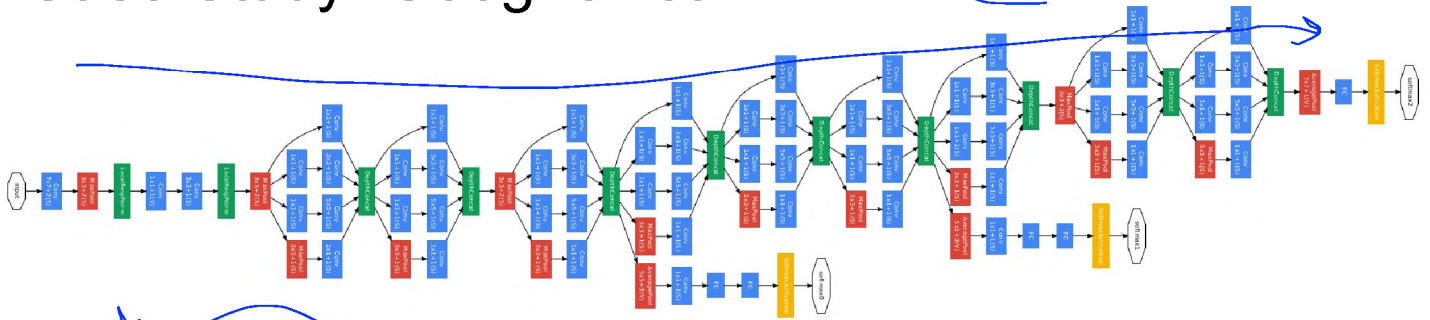
Lecture 7 - 69

27 Jan 2016

792

Case Study: GoogLeNet

[Szegedy et al., 2014]



Inception module

ILSVRC 2014 winner (6.7% top 5 error)

Fei-Fei Li & Andrej Karpathy & Justin Johnson

Lecture 7 - 75

27 Jan 2016

793

Case Study: ResNet

[He et al., 2015]

ILSVRC 2015 winner (3.6% top 5 error)

Microsoft Research

MSRA @ ILSVRC & COCO 2015 Competitions

- **1st places in all five main tracks**
 - ImageNet Classification: *"Ultra-deep"* (quote Yann) **152-layer** nets
 - ImageNet Detection: **16%** better than 2nd
 - ImageNet Localization: **27%** better than 2nd
 - COCO Detection: **11%** better than 2nd
 - COCO Segmentation: **12%** better than 2nd

*improvements are relative numbers

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition", arXiv 2015.

Slide from Kaiming He's recent presentation <https://www.youtube.com/watch?v=1PGLj-uKT1w>

Fei-Fei Li & Andrej Karpathy & Justin Johnson

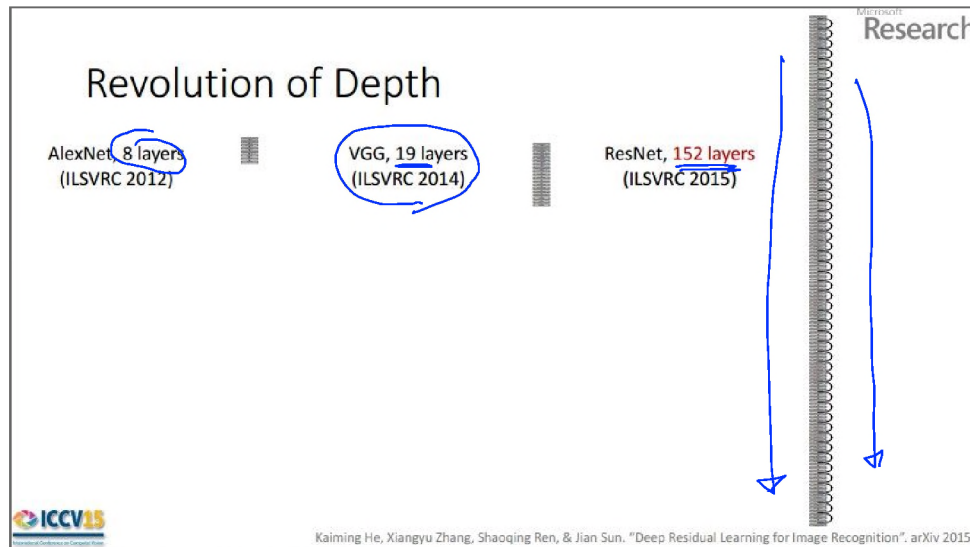
Lecture 7 - 77

27 Jan 2016

794

[He et al., 2015]

ILSVRC 2015 winner (3.6% top 5 error)



2-3 weeks of training
on 8 GPU machine

at runtime: faster
than a VGGNet!
(even though it has
8x more layers)

(slide from Kaiming He's recent presentation)

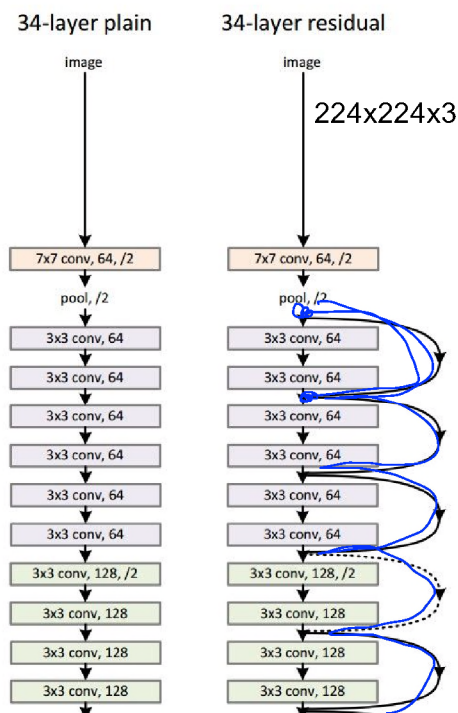
Fei-Fei Li & Andrej Karpathy & Justin Johnson

Lecture 7 - 80

27 Jan 2016

Case Study: ResNet

[He et al., 2015]



Fei-Fei Li & Andrej Karpathy & Justin Johnson

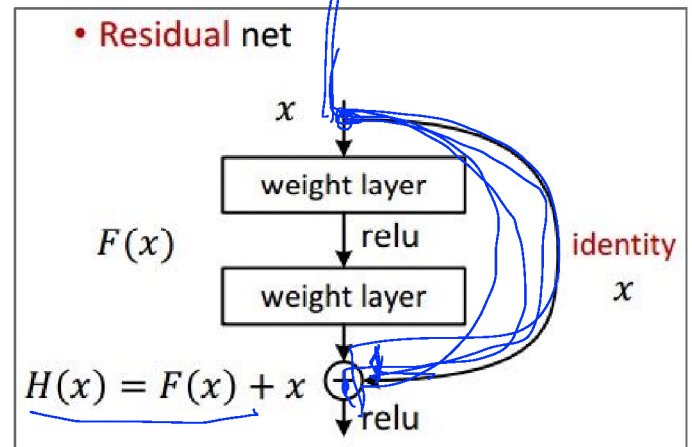
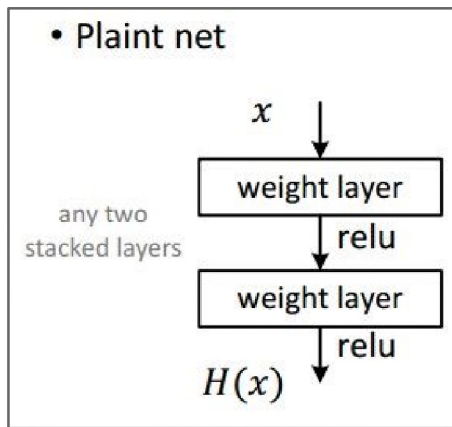
Lecture 7 - 81

27 Jan 2016

Case Study: ResNet

[He et al., 2015]

2016



Fei-Fei Li & Andrej Karpathy & Justin Johnson

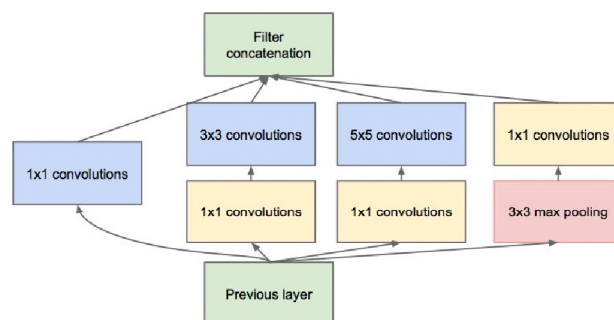
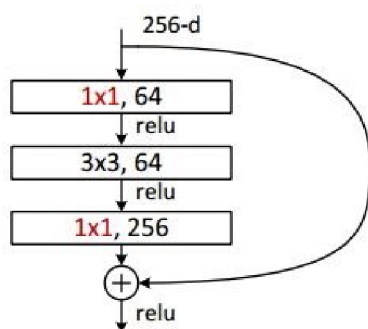
Lecture 7 - 82

27 Jan 2016

797

Case Study: ResNet

[He et al., 2015]



Fei-Fei Li & Andrej Karpathy & Justin Johnson

Lecture 7 - 85

27 Jan 2016

798

Convolutional Neural Networks for Sentence Classification

[Yoon Kim, 2014]

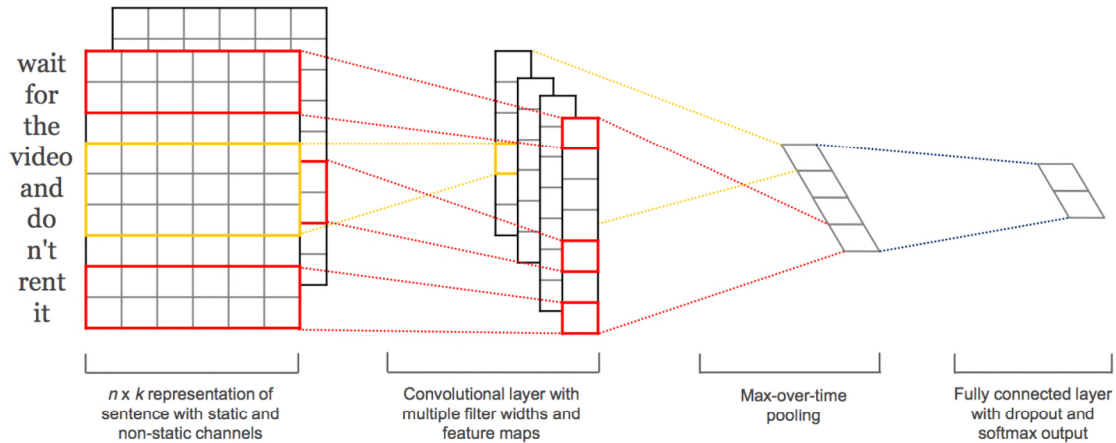
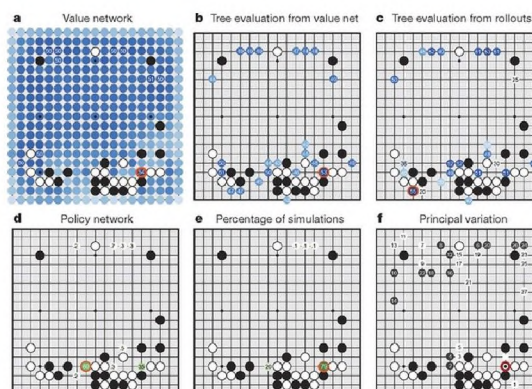


Figure 1: Model architecture with two channels for an example sentence.

799

Case Study Bonus: DeepMind's AlphaGo



800

The input to the policy network is a $19 \times 19 \times 48$ image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a 23×23 image, then convolves k filters of kernel size 5×5 with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequent hidden layers 2 to 12 zero pads the respective previous hidden layer into a 21×21 image, then convolves k filters of kernel size 3×3 with stride 1, again followed by a rectifier nonlinearity. The final layer convolves 1 filter of kernel size 1×1 with stride 1, with a different bias for each position, and applies a softmax function. The match version of AlphaGo used $k = 192$ filters; Fig. 2b and Extended Data Table 3 additionally show the results of training with $k = 128, 256$ and 384 filters.

policy network:

[19x19x48] Input

CONV1: 192 5×5 filters, stride 1, pad 2 => [19x19x192]

CONV2..12: 192 3×3 filters, stride 1, pad 1 => [19x19x192]

CONV: 1 1×1 filter, stride 1, pad 0 => [19x19] (*probability map of promising moves*)